

Relatório de Projeto Final

Beatriz Bezerra Rufino Ferreira Paiva e Fernando Praciano Guimarães

Edubot - 2026.1

1 Introdução

O objetivo deste projeto é programar o robô Sparki tal que ele seja capaz de realizar o labirinto proposto na Figura 1.

Durante o percurso, sempre que o Sparki encontrar um caminho sem saída, ele deverá verificar a presença de um objeto utilizando seus sensores e seu mecanismo de garra. Caso um objeto seja detectado e capturado, o robô deverá interromper imediatamente a exploração e retornar ao ponto inicial do labirinto. Caso contrário, deverá continuar a navegação até localizar o objeto.

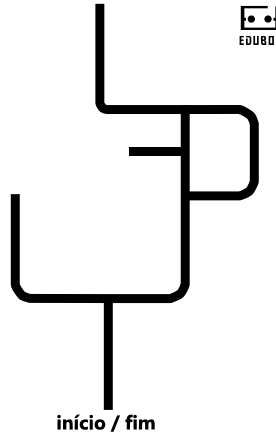


Figura 1: Percurso proposto

2 Implementação da solução

Uma vez que este problema apresenta um considerável grau de complexidade, separamos a implementação e os testes em etapas, para melhor documentar o desenvolvimento da solução. Essas etapas são:

1. Implementar um algoritmo seguidor de linha que seja capaz de chegar em todos os pontos sem saída.
2. Verificar se existe um objeto a ser capturado em cada ponto sem saída.
3. Fazer com que o Sparki retorne ao ponto de origem assim que o objeto seja capturado em qualquer um dos pontos sem saída.

Os testes foram realizados em labirintos do tamanho A2, um fornecido pelo laboratório e outro impresso por nós. No primeiro, as curvas foram transformadas em curvas de 90° e a linha foi coberta por fita preta para intensificar sua tonalidade.

2.1 Segue-linha adaptado para bifurcações

Em primeiro lugar, é preciso garantir que o Sparki siga a linha preta e explore o labirinto. Para isso, começamos por um algoritmo segue-linha já conhecido que se baseia em leituras dos sensores infravermelhos.

Devido às bifurcações, é necessário fazer algumas adaptações. O código apresentado na Listagem 1 usa uma ordem de prioridade esquerda > centro > direita para decidir qual caminho tomar em uma bifurcação.

```
1 #include <Sparki.h>
2
3 void setup() {
4 }
5
6 void loop() {
7     int lim = 500;
8
9     int lineLeft    = sparki.lineLeft();
10    int lineCenter   = sparki.lineCenter();
11    int lineRight    = sparki.lineRight();
12
13    if ( lineLeft < lim )
14    {
15        sparki.moveLeft();
16    }
17
18    else if ( lineCenter < lim )
19    {
20        sparki.moveForward();
21    }
22
23    else if ( lineRight < lim )
24    {
25        sparki.moveRight();
26    }
27
28    else
29    {
30        sparki.moveRight(180);
31    }
32
33    delay(100);
34 }
```

Listagem 1: Seguidor de linha adaptado

Este algoritmo permite que o Sparki alcance todos os pontos sem saída. Ao realizar os testes, percebemos algumas falhas na leitura dos sensores, principalmente em curvas e em pontos de encontro da fita que delimita o percurso. Apesar disso, pudemos verificar que a lógica estava correta.

Note-se que os erros iniciais nos levaram a imprimir uma cópia do labirinto em lona, para que não fosse necessário lidar com os erros nos pontos de encontro da fita e para que as curvas fossem suavizadas. Mesmo assim, o desempenho nas curvas não apresentou melhora considerável. Acreditamos que isso tenha ocorrido pois o Sparki tende a se descentralizar da linha logo após a primeira bifurcação.

Então, tentamos melhorar as condições para a leitura dos sensores engrossando as curvas do labirinto em lona com fita preta e fazendo extensões em alguns pontos, mas essa ideia também não foi bem-sucedida.

2.2 Detecção do objeto

Além disso, uma vez que é exigida a captura de um objeto durante a exploração do labirinto, devemos encontrar um meio de checar se esse objeto está diante do robô.

Nessa etapa, a principal limitação é a falta de um sensor específico na garra. Não é possível, por exemplo, em cada ponto sem saída fazer um teste simples de fechamento da garra e atestar se um objeto foi capturado. Entretanto, é possível verificar a presença de um objeto utilizando o sensor ultrassônico de maneira bastante simples.

O código apresentado na Listagem 2 usa a leitura do sensor ultrassônico e duas variáveis booleanas para verificar a presença e a captura do objeto.

```

1 #include <Sparki.h>
2
3 bool achouObjeto = false;
4 bool temObjeto = false;
5
6 void setup() {
7
8     sparki.clearLCD();
9     sparki.servo(SERVO_CENTER);
10    sparki.gripperOpen();
11    delay(5000);
12    sparki.gripperStop();
13
14 }
15
16 void loop() {
17
18     int lim = 500;
19     int lineLeft = sparki.lineLeft();
20     int lineCenter = sparki.lineCenter();
21     int lineRight = sparki.lineRight();
22     int cm = sparki.ping();
23
24     if (cm < 10 && cm > -1) // verifica se tem um objeto ao fim do caminho que o sparki entrou
25     {
26         temObjeto = true;
27         sparki.clearLCD();
28         sparki.print("achou");
29         sparki.updateLCD();
30     }
31
32     if (!achouObjeto) // caminho de ida (antes da captura do objeto)
33     {
34
35         if ( lineLeft < lim )
36         {
37             sparki.moveLeft();
38         }
39         else if ( lineCenter < lim )
40         {
41             sparki.moveForward();
42         }
43         else if ( lineRight < lim )
44         {
45             sparki.moveRight();
46         }
47         else // ponto sem saida
48         {
49             if (temObjeto)
50             {
51                 {
52                     sparki.moveStop();
53                     delay(1000);
54                     sparki.gripperClose();
55                     delay(5000);
56                     sparki.gripperStop();
57                     achouObjeto = true;
58                 }
59
60                 sparki.moveRight(180);
61             }
62
63             delay(100);
64         }
65     }
66 }

```

Listagem 2: Detector do objeto

2.3 Retorno à origem

Finalmente, para retornar ao ponto de origem, devemos encontrar uma forma de identificar em que saída o objeto foi encontrado.

Nesta etapa, a principal limitação foi a falta de familiaridade com a linguagem de programação. Mesmo sabendo que seria possível resolver este problema de maneira mais abrangente utilizando estruturas de dados, como árvores ou pilhas, optamos por uma solução específica baseada em condicionais para que fosse possível desenvolver o projeto no tempo limitado.

Como o algoritmo de exploração percorre o labirinto sempre na mesma ordem, foi possível associar cada ponto sem saída a um identificador numérico, conforme a Figura 2. Para isso, criamos um contador que é incrementado sempre que um ponto sem saída é alcançado. O código teste do contador está apresentado na Listagem 3.

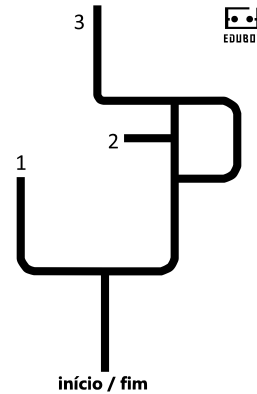


Figura 2: Becos numerados

```
1 #include <Sparki.h>
2
3 int pontoSemSaida = 0;
4
5 void setup() {
6 }
7
8 void loop() {
9     int lim = 500;
10    int lineLeft = sparki.lineLeft();
11    int lineCenter = sparki.lineCenter();
12    int lineRight = sparki.lineRight();
13
14    bool achouLinha = true;
15
16    if ( lineLeft < lim )
17    {
18        sparki.moveLeft();
19    }
20    else if ( lineCenter < lim )
21    {
22        sparki.moveForward();
23    }
24    else if ( lineRight < lim )
25    {
26        sparki.moveRight();
27    }
28    else // ponto sem saida
29    {
30        achouLinha = false;
31        sparki.moveRight(180);
32    }
33
34    if ( !achouLinha ) //contador
35    {
36        pontoSemSaida++;
37    }
38
39    sparki.clearLCD();
40    sparki.print(pontoSemSaida);
41    sparki.updateLCD();
42
43    delay(100);
44 }
```

Listagem 3: Contador

Após combinarmos os códigos da Listagem 2 e da Listagem 3, para cada caso, escolhemos uma ordem de preferência nas bifurcações que leva o Sparki diretamente ao ponto de origem. As estratégias utilizadas estão descritas a seguir:

Caso 1: a prioridade é esquerda > direita > centro. Observação: a prioridade à esquerda poderia estar em qualquer posição, pois em nenhum momento desse percurso de volta é preciso decidir entre esquerda e outra direção, mas colocamos a esquerda em primeiro por padrão.

Caso 2: este caso é um pouco mais complexo, pois somente as prioridades fariam com que o Sparki ficasse preso no loop. Pensamos em adicionar um segundo contador, para identificar as saídas à esquerda, porém não fomos capazes de fazer com que o contador registrasse essas leituras como eventos únicos, ele apenas registrava quantos milissegundos o sensor passou sobre a linha. Então, decidimos fazer com que o Sparki andasse para frente após sair da bifurcação por alguns centímetros e depois seguisse uma ordem de prioridades esquerda > centro > direita. O teste específico para esse caso está apresentado na Listagem 4.

Caso 3: a prioridade é esquerda > centro > direita.

```
1 #include <Sparki.h>
2 void setup (){
3 }
4
5 void loop (){
6
7 int lim = 500;
8 int lineLeft = sparki.lineLeft();
9 int lineCenter = sparki.lineCenter();
10 int lineRight = sparki.lineRight();
11
12 if (lineRight < lim && lineLeft < lim) // logo apos capturar o objeto, evita o loop
13 {
14     sparki.moveForward(5); // medimos experimentalmente para centralizar apos a curva
15     sparki.moveRight(90);
16     sparki.moveForward(10);
17 }
18
19 else if ( lineLeft < lim )
20 {
21     sparki.moveForward(5); // tambem para ajudar na centralizacao
22     sparki.moveLeft(90);
23 }
24 else if ( lineCenter < lim )
25 {
26     sparki.moveForward();
27 }
28
29 else if ( lineRight < lim )
30 {
31     sparki.moveRight();
32 }
33
34 else //voltou pro ponto de origem
35 {
36     sparki.moveStop();
37     sparki.gripperOpen();
38     delay(5000);
39     sparki.gripperStop();
40 }
41
42 }
```

Listagem 4: Retorno do caso 2

3 Resultados/Conclusão

O código completo está apresentado na Listagem 5. No geral, a solução proposta resolve bem este problema específico, mas pode falhar a depender da qualidade da leitura dos sensores, principalmente no que diz respeito ao contador de becos e aos caminhos de volta. A versão final completa da solução do problema e todos os códigos testados podem ser encontrados no repositório <https://github.com/bzrfp/pf-edubot>.

Para projetos futuros com percursos similares, recomendamos o uso de um labirinto em dimensões maiores, dessa forma os sensores não perderão a linha ao realizar o giro para sair dos becos sem saída. Recomendamos também o uso do ambiente de desenvolvimento Arduino IDE no lugar do SparkiDuino, pois este segundo apresentou diversos problemas na condução dos testes, travando sem motivo aparente e impossibilitando a escolha da porta em que o Sparki estava conectado, enquanto o primeiro, além de não apresentar os mesmos problemas, possui uma interface mais moderna e mais recursos. Para usá-lo, entretanto, deve-se adicionar a biblioteca do Sparki que pode ser encontrada em <https://github.com/ArcBotics/Sparki/tree/master>.

```
1 #include <Sparki.h>
2
3 int pontoSemSaida = 0;
4 bool achouObjeto = false;
5 bool temObjeto = false;
6
7 void setup() {
8
9     sparki.clearLCD();
10    sparki.servo(SERVO_CENTER);
11    sparki.gripperOpen();
12    delay(5000);
13    sparki.gripperStop();
14
15 }
16
17 void loop() {
18
19     int lim = 500;
20     int lineLeft = sparki.lineLeft();
21     int lineCenter = sparki.lineCenter();
22     int lineRight = sparki.lineRight();
23     int cm = sparki.ping();
24
25     bool achouLinha = true;
26
27     if (cm < 10 && cm > -1) // verifica se tem um objeto ao fim do caminho que o sparki entrou
28     {
29         temObjeto = true;
30         sparki.clearLCD();
31         sparki.print("achou");
32         sparki.updateLCD();
33     }
34
35     if (!achouObjeto) // caminho de ida (antes da captura do objeto)
36     {
37         if ( lineLeft < lim )
38         {
39             sparki.moveLeft();
40         }
41         else if ( lineCenter < lim )
42         {
43             sparki.moveForward();
44         }
45         else if ( lineRight < lim )
46         {
47             sparki.moveRight();
48         }
49         else // ponto sem saida
50         {
51             if (temObjeto)
52             {
53                 sparki.moveStop();
```

```

54     delay(1000);
55     sparki.gripperClose();
56     delay(5000);
57     sparki.gripperStop();
58     achouObjeto = true;
59 }
60 achouLinha = false;
61 sparki.moveRight(180);
62 }
63
64 //contador (para que identifiquemos em que saida o sparki encontrou o objeto)
65 if ( !achouLinha )
66 {
67     pontoSemSaida++;
68     sparki.clearLCD();
69     sparki.print(pontoSemSaida);
70     sparki.updateLCD();
71 }
72
73 delay(100);
74
75 }
76
77 else // caminho de volta
78 {
79
80     if ( pontoSemSaida == 1 )
81     {
82         if ( lineLeft < lim)
83         {
84             sparki.moveLeft();
85         }
86         else if ( lineRight < lim )
87         {
88             sparki.moveRight();
89         }
90         else if ( lineCenter < lim )
91         {
92             sparki.moveForward();
93         }
94         else //voltou pro ponto de origem
95         {
96             sparki.moveStop();
97             sparki.gripperOpen();
98             delay(5000);
99             sparki.gripperStop();
100         }
101     }
102
103     else if (pontoSemSaida == 2)
104     {
105         if (lineRight < lim && lineLeft < lim) // logo apos capturar o objeto, evita o loop
106         {
107             sparki.moveForward(5); // medimos experimentalmente para centralizar apos a curva
108             sparki.moveRight(90);
109             sparki.moveForward(10);
110         }
111         else if ( lineLeft < lim )
112         {
113             sparki.moveForward(5); // tambem para ajudar na centralizacao
114             sparki.moveLeft(90);
115         }
116         else if ( lineCenter < lim )
117         {
118             sparki.moveForward();
119         }
120         else if (lineRight < lim)
121         {
122             sparki.moveRight();
123         }
124         else //voltou pro ponto de origem

```

```

125     {
126         sparki.moveStop();
127         sparki.gripperOpen();
128         delay(5000);
129         sparki.gripperStop();
130     }
131 }
132
133 else if (pontoSemSaida == 3)
134 {
135     if ( lineLeft < lim )
136     {
137         sparki.moveLeft();
138     }
139     else if ( lineCenter < lim )
140     {
141         sparki.moveForward();
142     }
143     else if (lineRight < lim)
144     {
145         sparki.moveRight();
146     }
147     else //voltou pro ponto de origem
148     {
149         sparki.moveStop();
150         sparki.gripperOpen();
151         delay(5000);
152         sparki.gripperStop();
153     }
154 }
155
156 }
157
158 }

```

Listagem 5: Código final completo

4 Referências

1. <https://arcbotics.com/lessons/line-following/>
2. <https://arcbotics.com/lessons/object-retrieval/>